**INSTITUTO TÉCNICO RICALDONE**

Primer Año de Bachillerato · Desarrollo de Software

BTVDS 1.3 · Diseño de aplicaciones multimedia



# SIGMA

## Sistema Integral de Gestión de Mantenimiento de Aire acondicionado

**Proyecto de módulo · Problema 4**

Documentación técnica del sistema


**Integrantes**

[Nombre completo del integrante 1]

[Nombre completo del integrante 2]

[Nombre completo del integrante 3]

**Sección técnica:** [sección]

**Instructor:** Emerson González



San Salvador, octubre de 2026

# Índice


**1. Introducción ... 3**

**2. Planteamiento del problema y objetivos ... 4**

2.1 Problema ... 4

2.2 Objetivo general ... 4

2.3 Objetivos específicos ... 4

**3. Levantamiento de requerimientos ... 5**

3.1 Requerimientos funcionales ... 5

3.2 Requerimientos no funcionales ... 5

**4. Arquitectura del sistema ... 7**

4.1 Arquitectura Modelo-Vista ... 7

4.2 Estructura de carpetas ... 7

4.3 Tecnologías utilizadas ... 8

**5. Roles y permisos ... 9**

5.1 Alcance de los datos por rol ... 9

**6. Diagrama de casos de uso ... 10**

**7. Diagrama entidad-relación ... 11**

7.1 Relaciones ... 12

**8. Diccionario de datos ... 13**

8.1 Tabla Roles ... 13

8.2 Tabla Permisos ... 13

8.3 Tabla RolesPermisos ... 13

8.4 Tabla Usuarios ... 14

8.5 Tabla Clientes ... 14

8.6 Tabla Contratos ... 15

8.7 Tabla Equipos ... 15

8.8 Tabla Repuestos ... 16

8.9 Tabla Mantenimientos ... 16

8.10 Tabla MantenimientoRepuestos ... 17

8.11 Tabla RegistroActividad ... 17

8.12 Vistas ... 17

**9. Seguridad y manejo de excepciones ... 19**

9.1 Cifrado de contraseñas con BCrypt.Net ... 19

9.2 Otras medidas de seguridad ... 19

9.3 Manejo de excepciones ... 19

**10. Pantallas del sistema ... 21**

**11. Instalación y usuarios de prueba ... 25**

11.1 Requisitos ... 25

11.2 Pasos ... 25

11.3 Usuarios de prueba ... 25

# 1. Introducción

SIGMA (Sistema Integral de Gestión de Mantenimiento de Aire acondicionado) es una aplicación de escritorio para Windows desarrollada en C# con Windows Forms y SQL Server. Se construyó como proyecto de módulo para resolver el Problema 4: una empresa de mantenimiento de aire acondicionado que lleva sus contratos de mantenimiento preventivo, sus visitas de revisión, sus repuestos y la asignación de ingenieros en bitácoras físicas desorganizadas.

El sistema reemplaza esas bitácoras de papel por una base de datos centralizada. Con él, el administrador registra los contratos y los equipos de cada cliente, programa las visitas y les asigna un ingeniero; el ingeniero registra en una hoja de servicio lo que encontró, lo que hizo y los repuestos que consumió. Con eso queda un historial de mantenimiento de las instalaciones de cada cliente, que el administrador consulta cuando el cliente lo solicita.

Los clientes no usan el sistema: son las empresas dueñas de los contratos y de los equipos, y se registran como datos. Por eso el sistema tiene dos tipos de usuario, Administrador e Ingeniero.

El acceso está protegido por un inicio de sesión con contraseñas cifradas mediante la librería BCrypt.Net, y cada usuario ve únicamente las pantallas y los datos que le permite su rol. El programa sigue la arquitectura Modelo-Vista vista en clase: un proyecto contiene los datos y el acceso a la base (Modelo) y otro contiene los formularios (Vista).

Este documento reúne la documentación técnica del proyecto: el levantamiento de requerimientos, la arquitectura y la estructura de carpetas, los roles y permisos, el diagrama de casos de uso, el diagrama entidad-relación, el diccionario de datos, las medidas de seguridad y de manejo de excepciones, las pantallas principales y la guía de instalación.

# 2. Planteamiento del problema y objetivos

## 2.1 Problema

La empresa anota sus contratos, visitas y repuestos en bitácoras físicas. Eso provoca que no se sepa con certeza qué equipos cubre cada contrato, que una misma visita se agende dos veces o se olvide, que no quede registro confiable de lo que el ingeniero hizo en cada equipo, que el inventario de repuestos no coincida con lo que realmente se consumió y que, cuando un cliente pregunta por el historial de sus instalaciones, haya que buscarlo hoja por hoja.

## 2.2 Objetivo general

Desarrollar un sistema de escritorio que gestione los contratos de mantenimiento preventivo, las visitas de revisión, los repuestos y los ingenieros asignados de una empresa de mantenimiento de aire acondicionado, sustituyendo las bitácoras físicas por una base de datos segura y organizada.

## 2.3 Objetivos específicos

- Aplicar la arquitectura Modelo-Vista separando el acceso a datos de los formularios.
- Implementar un inicio de sesión con contraseñas cifradas con BCrypt.Net y dos roles: Administrador e Ingeniero.
- Construir el CRUD completo de equipos, mantenimientos, repuestos y contratos, además de clientes y usuarios.
- Controlar el acceso con permisos diferenciados por rol, administrables desde dentro del sistema.
- Registrar las hojas de servicio y descontar automáticamente del inventario los repuestos consumidos.
- Manejar los errores de la base de datos con Try-Catch y mostrarlos al usuario con mensajes claros.
- Entregar los scripts de la base de datos organizados y comentados, junto con esta documentación.

# 3. Levantamiento de requerimientos

## 3.1 Requerimientos funcionales

| Código | Requerimiento | Rol |
|---|---|---|
| RF-01 | Iniciar sesión con usuario y contraseña. La contraseña se compara contra su hash BCrypt. | Todos |
| RF-02 | Bloquear el botón de entrada durante 30 segundos después de 5 intentos fallidos. | Todos |
| RF-03 | Mostrar en el menú solo los módulos que permiten los permisos del rol. | Todos |
| RF-04 | Gestionar usuarios: crear, consultar, editar, desactivar y eliminar, asignándoles un rol. | Administrador |
| RF-05 | Asignar y quitar permisos a cada rol desde dentro del sistema. | Administrador |
| RF-06 | Gestionar clientes (CRUD): las empresas dueñas de los contratos. Se registran como datos; no tienen usuario. | Administrador |
| RF-07 | Gestionar contratos de mantenimiento preventivo (CRUD). | Administrador |
| RF-08 | Gestionar equipos de aire acondicionado (CRUD) asociados a un contrato. | Administrador |
| RF-09 | Gestionar repuestos (CRUD) con existencia, existencia mínima y aviso de existencia baja. | Administrador |
| RF-10 | Programar visitas de mantenimiento (CRUD) y asignarles un ingeniero, evitando dos visitas del mismo equipo el mismo día. | Administrador |
| RF-11 | Consultar las visitas que tiene asignadas. | Ingeniero |
| RF-12 | Registrar la hoja de servicio de una visita: estado, fecha, diagnóstico, trabajo realizado y observaciones. | Ingeniero |
| RF-13 | Registrar los repuestos consumidos en la visita y descontarlos del inventario. | Ingeniero |
| RF-14 | Consultar el historial de mantenimiento de las instalaciones de cada cliente, buscando por cliente, equipo, ingeniero o estado, y ver el detalle de cada hoja de servicio. | Administrador |
| RF-15 | Guardar en una bitácora quién hizo cada acción y cuándo, y permitir consultarla. | Administrador |

## 3.2 Requerimientos no funcionales

| Código | Requerimiento |
|---|---|
| RNF-01 | Arquitectura Modelo-Vista: la solución tiene un proyecto Modelo (datos) y un proyecto Vista (formularios). |
| RNF-02 | Las contraseñas se guardan cifradas con BCrypt.Net; nunca en texto plano. |
| RNF-03 | Todas las consultas usan parámetros (SqlParameter), lo que evita la inyección de SQL. |

| Código | Requerimiento |
|---|---|
| RNF-04 | Las operaciones con la base de datos se protegen con Try-Catch y los errores se informan con MessageBox. |
| RNF-05 | Los datos se validan en el formulario antes de enviarlos a la base de datos. |
| RNF-06 | La base de datos está normalizada, con llaves foráneas y restricciones CHECK y UNIQUE. |
| RNF-07 | Las operaciones que tocan varias tablas (hoja de servicio, permisos de un rol) se ejecutan dentro de una transacción. |
| RNF-08 | La interfaz es coherente: mismos colores, mismos botones y mismos mensajes en todas las pantallas. |
| RNF-09 | Plataforma: Windows 10 u 11, .NET Framework 4.8 y SQL Server Express. |
| RNF-10 | El código se entrega en un repositorio con estructura clara de carpetas. |

# 4. Arquitectura del sistema

## 4.1 Arquitectura Modelo-Vista

La solución SIGMA.sln está formada por dos proyectos. La Vista conoce al Modelo, pero el Modelo no conoce a la Vista: así el acceso a datos no depende de ningún formulario.

| Proyecto | Tipo | Responsabilidad |
|---|---|---|
| **Modelo** | Biblioteca de clases | Conexión a SQL Server, clases de entidad (Usuario, Contrato, Equipo...) y clases de datos (UsuarioDatos, ContratoDatos...) con las consultas INSERT, SELECT, UPDATE y DELETE. También el cifrado BCrypt, la sesión actual y la traducción de errores de SQL. |
| **Vista** | Aplicación Windows Forms | Formularios, validación de lo que escribe el usuario, mensajes (MessageBox) y estilos. Los formularios nunca escriben SQL: llaman a las clases de datos del Modelo. |

Cada entidad tiene dos clases en el Modelo: la clase con sus propiedades (por ejemplo **Contrato**) y la clase estática con sus operaciones sobre la base (por ejemplo **ContratoDatos**, con los métodos Listar, Insertar, Actualizar y Eliminar). Cada formulario de la Vista corresponde a un módulo del menú.

## 4.2 Estructura de carpetas

```
SIGMA/
├── SIGMA.sln                    Solución de Visual Studio
├── README.md                    Guía rápida de instalación
├── Documentacion/               Este documento, diagramas y capturas
├── Modelo/                      PROYECTO MODELO (biblioteca de clases)
│   ├── ConexionDB/              Conexion.cs: único punto de conexión a SQL Server
│   ├── Database/                SIGMA_Script.sql y CrearBaseDeDatos.cmd
│   └── Modelo/                  Entidades y clases de datos
│       ├── Usuario.cs, Rol.cs, Permiso.cs, Cliente.cs, Contrato.cs,
│       │   Equipo.cs, Repuesto.cs, Mantenimiento.cs, RepuestoUsado.cs ...
│       ├── UsuarioDatos.cs, RolDatos.cs, PermisoDatos.cs, ClienteDatos.cs,
│       │   ContratoDatos.cs, EquipoDatos.cs, RepuestoDatos.cs,
│       │   MantenimientoDatos.cs, RegistroActividadDatos.cs
│       └── Clave.cs (BCrypt), Sesion.cs, Errores.cs, Nulos.cs, Textos.cs
└── Vista/                       PROYECTO VISTA (Windows Forms)
    ├── Program.cs               Punto de entrada: login y ventana principal
    ├── App.config               Cadena de conexión
    ├── Formularios/             FrmLogin, FrmMain, FrmInicio, FrmContratos,
    │                            FrmEquipos, FrmMantenimientos, FrmHojaServicio,
    │                            FrmRepuestos, FrmClientes, FrmUsuarios,
    │                            FrmRoles, FrmBitacora
    ├── Estilos/                 Tema.cs, EstiloGrid.cs, EstiloBotones.cs, Combos.cs
    ├── Utilidades/              Mensajes.cs, Validaciones.cs, Filtro.cs, Ventanas.cs
    └── Resources/               sigma.ico
```

## 4.3 Tecnologías utilizadas

| Tecnología | Uso en el proyecto |
|---|---|
| C# y .NET Framework 4.8 | Lenguaje y plataforma de los dos proyectos. |
| Windows Forms | Formularios de la capa Vista. |
| SQL Server Express | Motor de la base de datos SIGMA. |
| ADO.NET (System.Data.SqlClient) | Conexión, comandos parametrizados y transacciones. |
| BCrypt.Net-Next 4.0.3 | Cifrado y verificación de contraseñas. |
| Guna.UI2.WinForms 2.0.4.8 | Controles visuales (botones, cajas de texto, combos). |
| Git y GitHub | Control de versiones y repositorio del proyecto. |

# 5. Roles y permisos

El control de acceso se guarda en tres tablas: **Roles** (los tipos de usuario), **Permisos** (las acciones que el sistema sabe controlar) y **RolesPermisos** (qué permisos tiene cada rol). Cada usuario tiene un rol; al iniciar sesión el programa carga los permisos de ese rol y con ellos decide qué botones del menú mostrar y qué acciones permitir.

El sistema tiene dos roles: **Administrador** e **Ingeniero**. Los clientes no tienen rol ni usuario, porque nunca interactúan directamente con el sistema. El administrador puede cambiar los permisos del rol Ingeniero desde la pantalla **Roles y permisos**. El rol Administrador siempre conserva todos los permisos, para que el sistema nunca se quede sin alguien que pueda administrarlo. La tabla siguiente muestra los permisos con los que se instala el sistema.

| Módulo | Permiso | Administrador | Ingeniero |
|---|---|---|---|
| Contratos | Consultar contratos | Sí | — |
| Contratos | Crear, editar y eliminar contratos | Sí | — |
| Equipos | Consultar equipos | Sí | Sí |
| Equipos | Crear, editar y eliminar equipos | Sí | — |
| Repuestos | Consultar repuestos | Sí | Sí |
| Repuestos | Crear, editar y eliminar repuestos | Sí | — |
| Mantenimientos | Consultar el historial de mantenimientos | Sí | Sí |
| Mantenimientos | Programar visitas y asignar ingenieros | Sí | — |
| Mantenimientos | Registrar hojas de servicio y repuestos consumidos | Sí | Sí |
| Clientes | Crear, editar y eliminar clientes corporativos | Sí | — |
| Administración | Gestionar usuarios, roles y permisos | Sí | — |
| Administración | Consultar la bitácora de actividad | Sí | — |

## 5.1 Alcance de los datos por rol

Además de los permisos, el sistema limita qué registros ve cada rol:

- **Ingeniero:** solo ve las visitas que tiene asignadas y solo puede registrar la hoja de servicio de esas visitas. Consulta equipos y repuestos, sin modificarlos.
- **Administrador:** ve y gestiona todos los registros, incluidos usuarios, roles, permisos y bitácora.

# 6. Diagrama de casos de uso

El diagrama muestra los dos actores del sistema y lo que cada uno puede hacer. Ambos inician sesión; a partir de ahí cada actor accede a los casos de uso de su rol. Como el Administrador tiene todos los permisos, también puede realizar los casos de uso del Ingeniero.

El cliente no aparece como actor porque nunca interactúa directamente con el sistema: es el administrador quien registra sus datos, sus contratos y sus equipos, y quien consulta el historial de mantenimiento cuando el cliente lo pide.

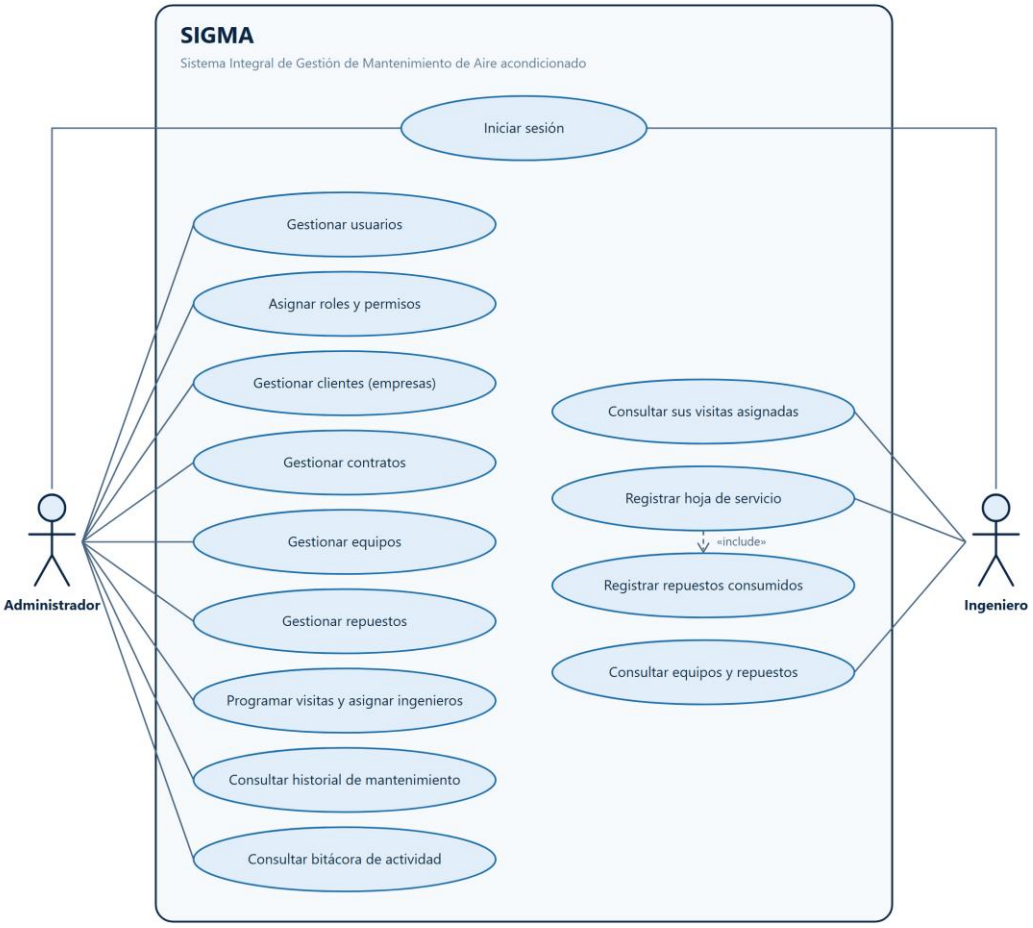



*Figura 1. Diagrama de casos de uso de SIGMA*

| Actor | Casos de uso |
|---|---|
| **Administrador** | Iniciar sesión · Gestionar usuarios · Asignar roles y permisos · Gestionar clientes (empresas) · Gestionar contratos · Gestionar equipos · Gestionar repuestos · Programar visitas y asignar ingenieros · Consultar historial de mantenimiento · Consultar bitácora de actividad |
| **Ingeniero** | Iniciar sesión · Consultar sus visitas asignadas · Registrar hoja de servicio (incluye registrar los repuestos consumidos) · Consultar equipos y repuestos |

# 7. Diagrama entidad-relación

La base de datos SIGMA tiene once tablas. Cuatro son de seguridad (Roles, Permisos, RolesPermisos y Usuarios), seis son del negocio (Clientes, Contratos, Equipos, Repuestos, Mantenimientos y MantenimientoRepuestos) y una es la bitácora (RegistroActividad).




*Figura 2. Diagrama entidad-relación de la base de datos SIGMA*

## 7.1 Relaciones

| Tabla padre | Tabla hija | Tipo | Llave foránea | Significado |
|---|---|---|---|---|
| Clientes | Contratos | 1 a N | IdCliente | Un cliente puede firmar muchos contratos. |
| Contratos | Equipos | 1 a N | IdContrato | Un contrato cubre muchos equipos. |
| Equipos | Mantenimientos | 1 a N | IdEquipo | Un equipo recibe muchas visitas de mantenimiento. |
| Mantenimientos | MantenimientoRepuestos | 1 a N | IdMantenimiento | En una visita se consumen muchos repuestos. |
| Permisos | RolesPermisos | 1 a N | IdPermiso | Un permiso puede estar asignado a muchos roles. |
| Repuestos | MantenimientoRepuestos | 1 a N | IdRepuesto | Un repuesto se consume en muchas visitas. |
| Roles | RolesPermisos | 1 a N | IdRol | Un rol tiene muchos permisos asignados. |
| Roles | Usuarios | 1 a N | IdRol | Un rol lo tienen muchos usuarios; cada usuario tiene un solo rol. |
| Usuarios | Mantenimientos | 1 a N | IdIngeniero | Un ingeniero atiende muchas visitas; cada visita tiene un ingeniero. |
| Usuarios | RegistroActividad | 1 a N | IdUsuario | Un usuario genera muchas acciones en la bitácora. |

Las relaciones de muchos a muchos se resuelven con tablas intermedias: **RolesPermisos** une Roles con Permisos, y **MantenimientoRepuestos** une Mantenimientos con Repuestos (guardando además la cantidad y el precio del momento).

# 8. Diccionario de datos

Para cada tabla se indica el nombre del campo, su tipo de dato, si admite valores nulos, si es llave primaria (PK) o foránea (FK) y su descripción con las restricciones que tiene.

## 8.1 Tabla Roles

Tipos de usuario del sistema. Son dos: Administrador e Ingeniero.

| Campo | Tipo de dato | Nulo | Llave | Descripción |
|---|---|---|---|---|
| **IdRol** | INT | No | PK | Identificador del rol (autonumérico). |
| **Nombre** | VARCHAR(30) | No | | Nombre del rol. No se repite (UNIQUE). |
| **Descripcion** | NVARCHAR(150) | No | | Para qué sirve el rol. |

## 8.2 Tabla Permisos

Acciones que el sistema sabe controlar. Cada fila es un permiso que se puede dar o quitar a un rol.

| Campo | Tipo de dato | Nulo | Llave | Descripción |
|---|---|---|---|---|
| **IdPermiso** | INT | No | PK | Identificador del permiso (autonumérico). |
| **Clave** | VARCHAR(40) | No | | Identificador que revisa el programa, por ejemplo CONTRATOS_GESTIONAR. No se repite (UNIQUE). |
| **Nombre** | NVARCHAR(80) | No | | Texto del permiso que ve el administrador. |
| **Modulo** | NVARCHAR(30) | No | | Módulo o pantalla a la que pertenece el permiso. |

## 8.3 Tabla RolesPermisos

Permisos asignados a cada rol. Resuelve la relación de muchos a muchos entre Roles y Permisos.

| Campo | Tipo de dato | Nulo | Llave | Descripción |
|---|---|---|---|---|
| **IdRol** | INT | No | PK, FK → Roles | Rol que recibe el permiso. |
| **IdPermiso** | INT | No | PK, FK → Permisos | Permiso asignado al rol. La llave primaria compuesta impide asignarlo dos veces. |

## 8.4 Tabla Usuarios

Personas que pueden iniciar sesión en el sistema: administradores e ingenieros.

| Campo | Tipo de dato | Nulo | Llave | Descripción |
|---|---|---|---|---|
| **IdUsuario** | INT | No | PK | Identificador del usuario (autonumérico). |
| **Nombre** | NVARCHAR(100) | No | | Nombre completo de la persona. |
| **NombreUsuario** | VARCHAR(30) | No | | Nombre con el que inicia sesión. No se repite (UNIQUE). |
| **Contrasena** | VARCHAR(100) | No | | Hash de la contraseña generado con BCrypt (60 caracteres). Nunca se guarda el texto real. |
| **IdRol** | INT | No | FK → Roles | Rol del usuario; de él dependen sus permisos. |
| **Correo** | VARCHAR(100) | Sí | | Correo electrónico (opcional). |
| **Activo** | BIT | No | | 1 = puede iniciar sesión; 0 = desactivado. Valor por defecto: 1. |
| **FechaCreacion** | DATETIME | No | | Fecha y hora en que se creó el usuario. Valor por defecto: GETDATE(). |
| **UltimoAcceso** | DATETIME | Sí | | Fecha y hora del último inicio de sesión. |

## 8.5 Tabla Clientes

Empresas que contratan el servicio de mantenimiento. Solo se guardan sus datos: los clientes no tienen usuario ni entran al sistema.

| Campo | Tipo de dato | Nulo | Llave | Descripción |
|---|---|---|---|---|
| **IdCliente** | INT | No | PK | Identificador del cliente (autonumérico). |
| **Nombre** | NVARCHAR(100) | No | | Razón social de la empresa. No se repite (UNIQUE). |
| **Nit** | VARCHAR(17) | No | | Número de identificación tributaria, con formato 0000-000000-000-0. No se repite (UNIQUE). |
| **Contacto** | NVARCHAR(100) | No | | Persona encargada en la empresa. |
| **Telefono** | VARCHAR(9) | No | | Teléfono con formato 0000-0000. |
| **Correo** | VARCHAR(100) | Sí | | Correo de contacto (opcional). |
| **Direccion** | NVARCHAR(200) | No | | Dirección de la empresa. |

## 8.6 Tabla Contratos

Contratos de mantenimiento preventivo firmados con un cliente. El programa los muestra con el código CT-0001.

| Campo | Tipo de dato | Nulo | Llave | Descripción |
|---|---|---|---|---|
| **IdContrato** | INT | No | PK | Identificador del contrato (autonumérico). |
| **IdCliente** | INT | No | FK → Clientes | Cliente que firmó el contrato. |
| **FechaInicio** | DATE | No | | Inicio de la vigencia. |
| **FechaFin** | DATE | No | | Fin de la vigencia. CHECK: debe ser posterior a FechaInicio. |
| **Frecuencia** | VARCHAR(15) | No | | Cada cuánto se hace la visita preventiva. CHECK: Mensual, Bimestral, Trimestral, Semestral o Anual. |
| **MontoMensual** | DECIMAL(10,2) | No | | Monto mensual del contrato en dólares. CHECK: mayor que 0. |
| **Estado** | VARCHAR(15) | No | | CHECK: Vigente, Suspendido o Finalizado. |
| **Descripcion** | NVARCHAR(300) | Sí | | Alcance del contrato (opcional). |

## 8.7 Tabla Equipos

Equipos de aire acondicionado instalados donde el cliente y cubiertos por un contrato.

| Campo | Tipo de dato | Nulo | Llave | Descripción |
|---|---|---|---|---|
| **IdEquipo** | INT | No | PK | Identificador del equipo (autonumérico). |
| **IdContrato** | INT | No | FK → Contratos | Contrato que cubre al equipo; a través de él se sabe de qué cliente es. |
| **NumeroSerie** | VARCHAR(30) | No | | Número de serie de la placa del equipo. No se repite (UNIQUE). |
| **Marca** | NVARCHAR(40) | No | | Marca del equipo. |
| **Modelo** | NVARCHAR(40) | No | | Modelo del equipo. |
| **Tipo** | VARCHAR(20) | No | | CHECK: Mini Split, Cassette, Piso-Techo, Central, Ventana o Chiller. |
| **CapacidadBTU** | INT | No | | Capacidad de enfriamiento en BTU. CHECK: mayor que 0. |
| **Ubicacion** | NVARCHAR(150) | No | | Lugar de la instalación del cliente donde está el equipo. |
| **FechaInstalacion** | DATE | No | | Fecha en que se instaló el equipo. |

| Campo | Tipo de dato | Nulo | Llave | Descripción |
|---|---|---|---|---|
| **Estado** | VARCHAR(20) | No | | CHECK: Operativo, En mantenimiento o Fuera de servicio. |

## 8.8 Tabla Repuestos

Inventario de piezas que los ingenieros consumen en las visitas. El programa los muestra con el código RP-0001.

| Campo | Tipo de dato | Nulo | Llave | Descripción |
|---|---|---|---|---|
| **IdRepuesto** | INT | No | PK | Identificador del repuesto (autonumérico). |
| **Nombre** | NVARCHAR(80) | No | | Nombre del repuesto. No se repite (UNIQUE). |
| **Categoria** | NVARCHAR(20) | No | | CHECK: Eléctrico, Mecánico, Refrigeración, Filtración u Otro. |
| **Existencia** | INT | No | | Unidades disponibles en bodega. CHECK: mayor o igual que 0. |
| **StockMinimo** | INT | No | | Cantidad a partir de la cual se avisa existencia baja. CHECK: mayor o igual que 0. |
| **PrecioUnitario** | DECIMAL(10,2) | No | | Precio del repuesto en dólares. CHECK: mayor que 0. |

## 8.9 Tabla Mantenimientos

Visitas de revisión. Cada fila es también la hoja de servicio que llena el ingeniero. El programa las muestra con el código MT-0001.

| Campo | Tipo de dato | Nulo | Llave | Descripción |
|---|---|---|---|---|
| **IdMantenimiento** | INT | No | PK | Identificador de la visita (autonumérico). |
| **IdEquipo** | INT | No | FK → Equipos | Equipo que se revisa. |
| **IdIngeniero** | INT | No | FK → Usuarios | Usuario con rol Ingeniero asignado a la visita. |
| **Tipo** | VARCHAR(12) | No | | CHECK: Preventivo o Correctivo. |
| **FechaProgramada** | DATE | No | | Fecha en que debe hacerse la visita. |
| **FechaRealizada** | DATE | Sí | | Fecha en que se hizo. Se llena al completar la hoja de servicio. |
| **Estado** | VARCHAR(15) | No | | CHECK: Programado, En proceso, Completado o Cancelado. |

| Campo | Tipo de dato | Nulo | Llave | Descripción |
|---|---|---|---|---|
| **Diagnostico** | NVARCHAR(500) | Sí | | Lo que el ingeniero encontró al revisar el equipo. |
| **TrabajoRealizado** | NVARCHAR(500) | Sí | | Lo que el ingeniero le hizo al equipo. |
| **Observaciones** | NVARCHAR(300) | Sí | | Indicaciones del administrador o recomendaciones del ingeniero. |

## 8.10 Tabla MantenimientoRepuestos

Repuestos consumidos en cada visita (detalle de la hoja de servicio). Resuelve la relación de muchos a muchos entre Mantenimientos y Repuestos.

| Campo | Tipo de dato | Nulo | Llave | Descripción |
|---|---|---|---|---|
| **IdDetalle** | INT | No | PK | Identificador del detalle (autonumérico). |
| **IdMantenimiento** | INT | No | FK → Mantenimientos | Visita en la que se consumió el repuesto. |
| **IdRepuesto** | INT | No | FK → Repuestos | Repuesto consumido. UNIQUE junto con IdMantenimiento: no se repite en la misma visita. |
| **Cantidad** | INT | No | | Unidades consumidas. CHECK: mayor que 0. |
| **PrecioUnitario** | DECIMAL(10,2) | No | | Precio del repuesto en el momento en que se consumió. |

## 8.11 Tabla RegistroActividad

Bitácora del sistema: guarda quién hizo cada acción y cuándo. Reemplaza la bitácora física.

| Campo | Tipo de dato | Nulo | Llave | Descripción |
|---|---|---|---|---|
| **IdRegistro** | INT | No | PK | Identificador del registro (autonumérico). |
| **IdUsuario** | INT | No | FK → Usuarios | Usuario que hizo la acción. |
| **Accion** | NVARCHAR(200) | No | | Descripción de la acción realizada. |
| **Fecha** | DATETIME | No | | Fecha y hora de la acción. |

## 8.12 Vistas

Las vistas unen las tablas para que el programa lea los nombres (cliente, rol, ingeniero) sin repetir los INNER JOIN en cada consulta.

| Vista | Qué devuelve |
|---|---|
| **VistaUsuarios** | Usuarios con el nombre de su rol. |
| **VistaRolesPermisos** | Permisos de cada rol con sus nombres y claves. |
| **VistaContratos** | Contratos con el nombre del cliente y la cantidad de equipos que cubren. |
| **VistaEquipos** | Equipos con el cliente al que pertenecen (a través del contrato). |
| **VistaMantenimientos** | Visitas con los datos del equipo, el cliente, el ingeniero y el costo total de repuestos. |
| **VistaRepuestosUsados** | Repuestos consumidos en cada visita, con su nombre y subtotal. |
| **VistaActividad** | Bitácora con el usuario que hizo cada acción. |

# 9. Seguridad y manejo de excepciones

## 9.1 Cifrado de contraseñas con BCrypt.Net

La clase **Clave** del Modelo es la única que usa la librería BCrypt.Net. Al crear un usuario o cambiarle la contraseña, el método Cifrar genera un hash con costo 11 y una sal aleatoria; eso es lo único que llega a la base de datos. Al iniciar sesión, el método Coincide verifica la contraseña escrita contra el hash guardado. El hash no se puede revertir: ni el administrador ni quien lea la base puede conocer la contraseña.

```
public static string Cifrar(string clave)
{
    return BCrypt.Net.BCrypt.HashPassword(clave, Costo);     // Costo = 11
}

public static bool Coincide(string clave, string guardado)
{
    return BCrypt.Net.BCrypt.Verify(clave, guardado);
}
```

## 9.2 Otras medidas de seguridad

- **Contraseña segura:** mínimo 8 caracteres, con letras y números.
- **Mismo mensaje de error** para usuario inexistente y contraseña incorrecta, para no revelar qué usuarios existen.
- **Bloqueo temporal:** tras 5 intentos fallidos el inicio de sesión se bloquea 30 segundos.
- **Consultas parametrizadas:** ningún dato del usuario se concatena en el SQL.
- **Autenticación de Windows** en la conexión a SQL Server: el programa no guarda ninguna contraseña de la base de datos.
- **Doble control de permisos:** el menú esconde lo que el rol no puede usar y, además, cada botón Guardar o Eliminar vuelve a revisar el permiso.
- **Sesión siempre actualizada:** al abrir cada módulo se vuelve a leer el usuario; si fue desactivado o su rol perdió permisos, el cambio se aplica de inmediato.
- **Protección del administrador:** no se puede eliminar ni desactivar al último Administrador activo, ni un usuario puede eliminarse a sí mismo.

## 9.3 Manejo de excepciones

Todas las operaciones con la base de datos están dentro de un bloque Try-Catch. En el Modelo, cada método de las clases de datos atrapa la SqlException y la convierte en un mensaje en español con un código propio (clase **Errores**). En la Vista, cada formulario atrapa esa excepción y la muestra con un MessageBox (clase **Mensajes**), sin que el programa se cierre.

```
// Modelo: ContratoDatos.Eliminar
try
{
    using (SqlConnection con = Conexion.Abrir())
    using (SqlCommand cmd = new SqlCommand("DELETE FROM Contratos WHERE IdContrato = @Id", con))
    {
        cmd.Parameters.AddWithValue("@Id", idContrato);
        return cmd.ExecuteNonQuery();
    }
}
catch (SqlException ex)
{
    throw new Exception(Errores.Texto(ex), ex);
}

// Vista: FrmContratos.btnEliminar_Click
try
{
    ContratoDatos.Eliminar(c.IdContrato);
    Mensajes.Info("Contrato eliminado.");
}
catch (Exception ex)
{
    Mensajes.Fallo("No se pudo eliminar el contrato.", ex);
}
```

Además, Program.cs registra dos manejadores globales para cualquier error no previsto, de modo que la aplicación nunca termina con una ventana de error de Windows.

| Código | Cuándo aparece |
|---|---|
| ERR-SQL-001 | No se pudo conectar con SQL Server (servicio apagado o servidor incorrecto). |
| ERR-SQL-002 | Se intentó repetir un dato único (usuario, NIT, número de serie...). |
| ERR-SQL-003 | Se intentó eliminar un registro que otro usa (llave foránea). |
| ERR-SQL-004 | La consulta pide una tabla o columna que no existe. |
| ERR-SQL-005 | La base de datos tardó demasiado en responder. |
| ERR-SQL-007 | La operación rompe una regla del negocio (por ejemplo, no hay existencia suficiente de un repuesto). |
| ERR-SQL-008 | La base de datos SIGMA no existe o el usuario de Windows no tiene permiso. |
| ERR-VAL-001 a 003 | Validación del formulario: campo obligatorio vacío, formato incorrecto o número fuera de rango. |
| ERR-APP-001 / 002 | Error inesperado de la aplicación. |

# 10. Pantallas del sistema

Todas las pantallas comparten los mismos colores, tipografía y botones. Las tablas tienen un buscador; los formularios marcan el campo con error y explican qué corregir.

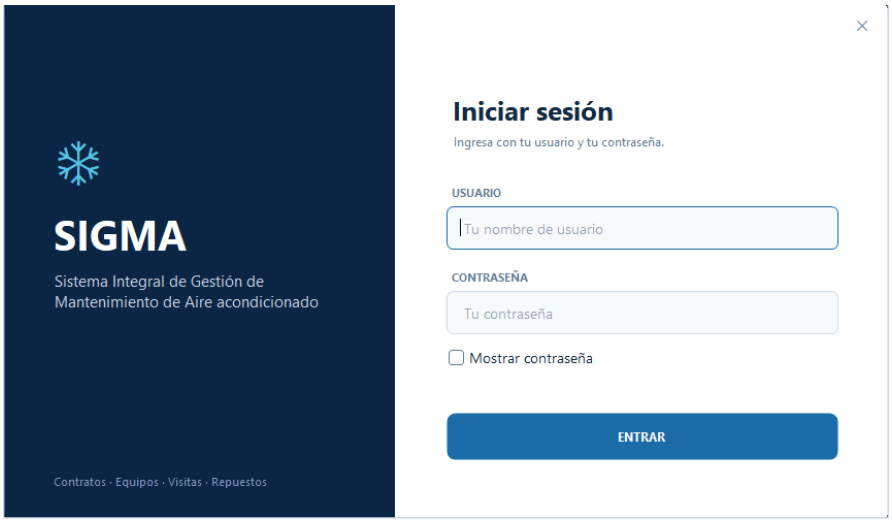



*Figura 3. Inicio de sesión*

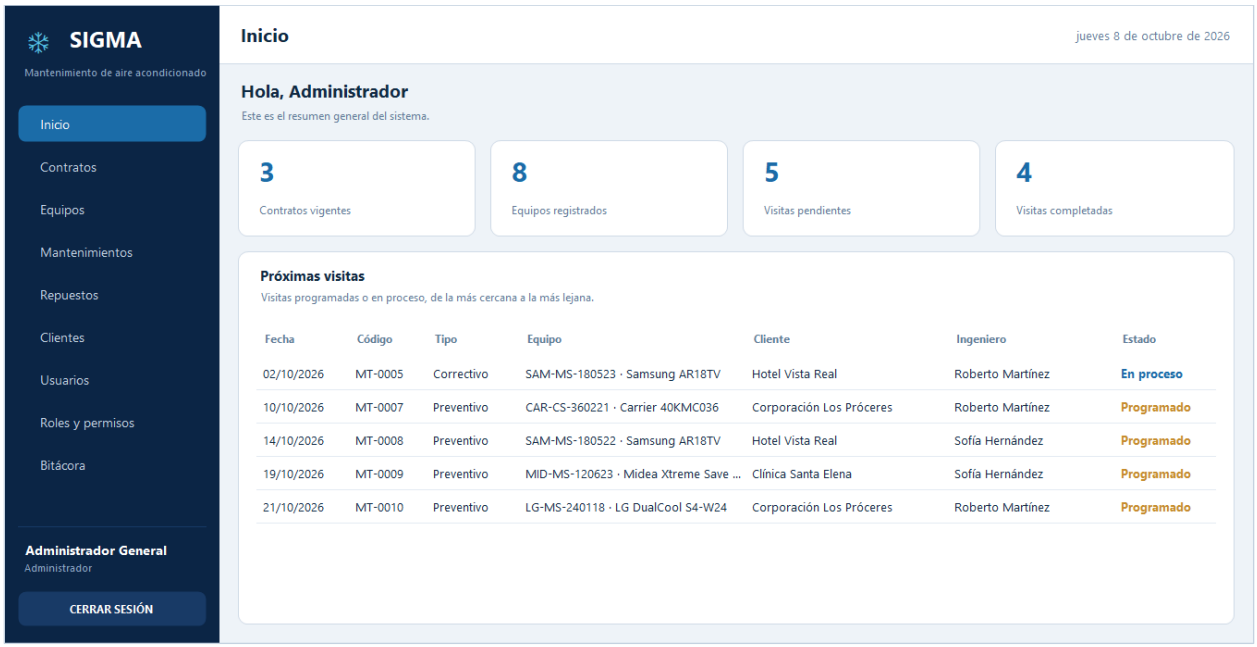



*Figura 4. Pantalla de inicio del Administrador: menú completo y resumen del sistema*

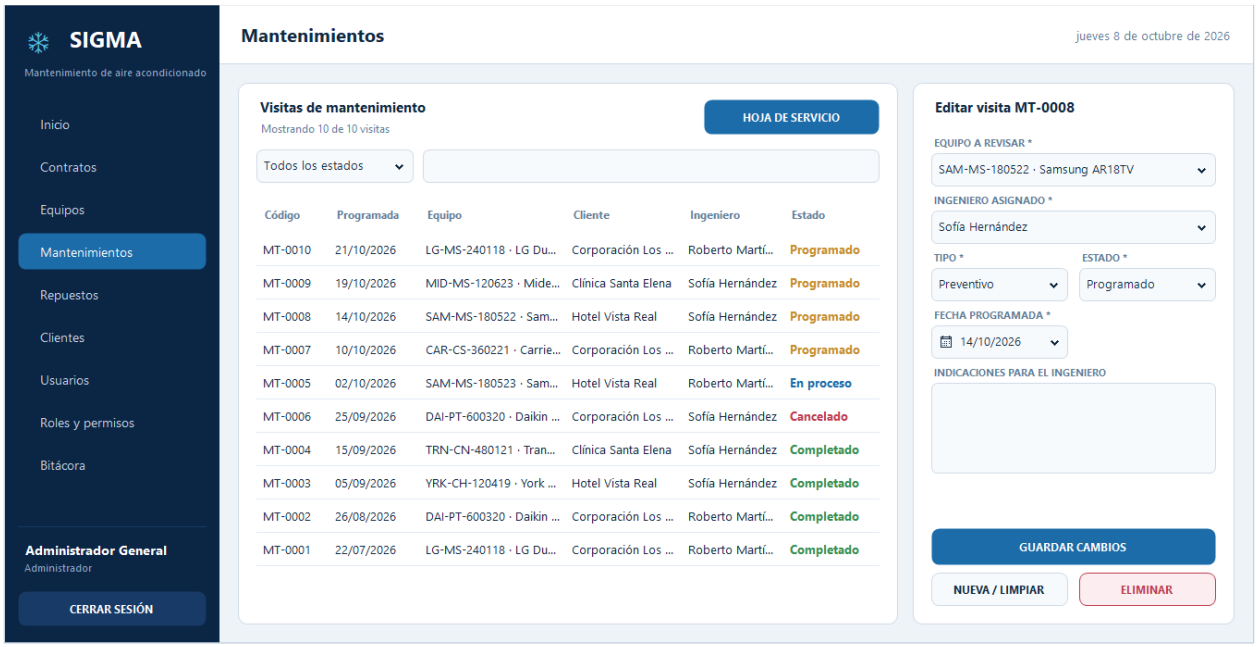



*Figura 5. Administrador: programación de visitas, asignación del ingeniero e historial de mantenimiento por cliente*

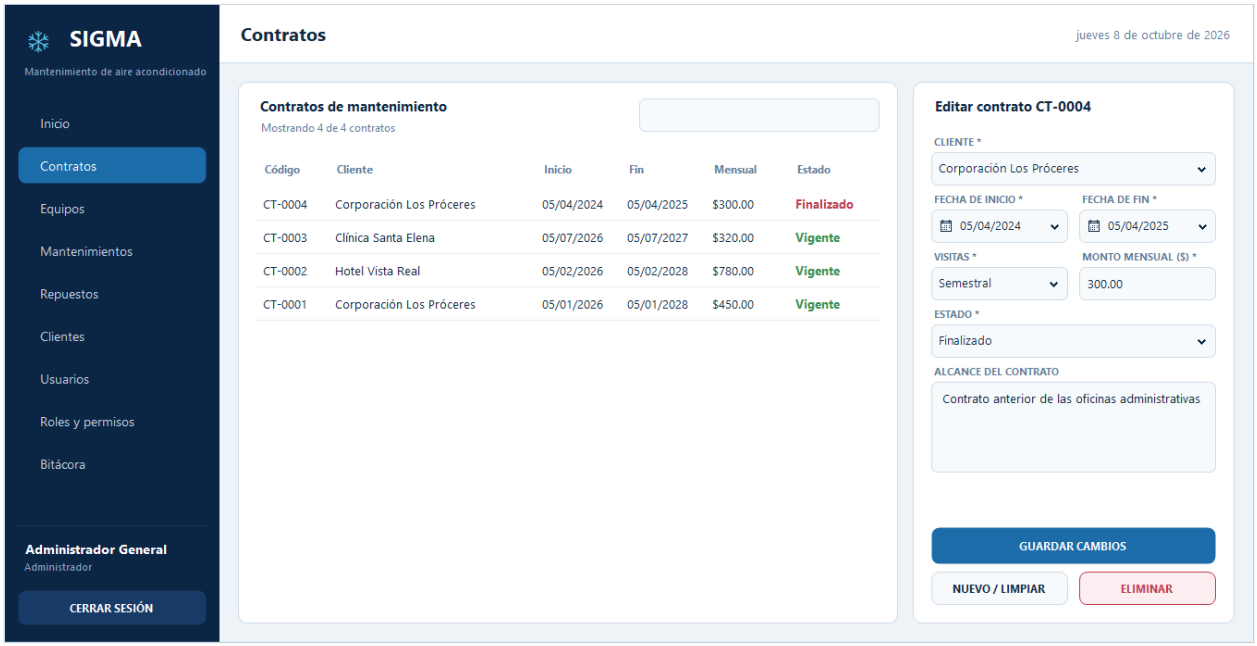



*Figura 6. Administrador: gestión de contratos (el mismo diseño se usa en equipos, repuestos y clientes)*

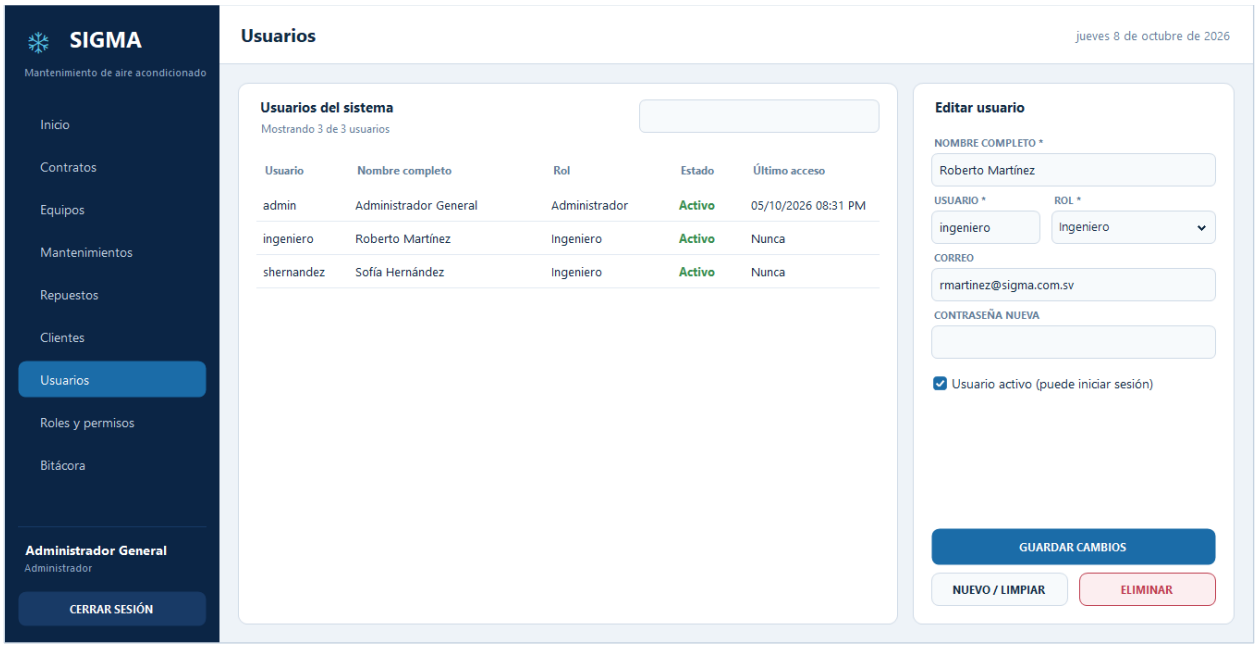



*Figura 7. Administrador: gestión de usuarios con asignación de rol*

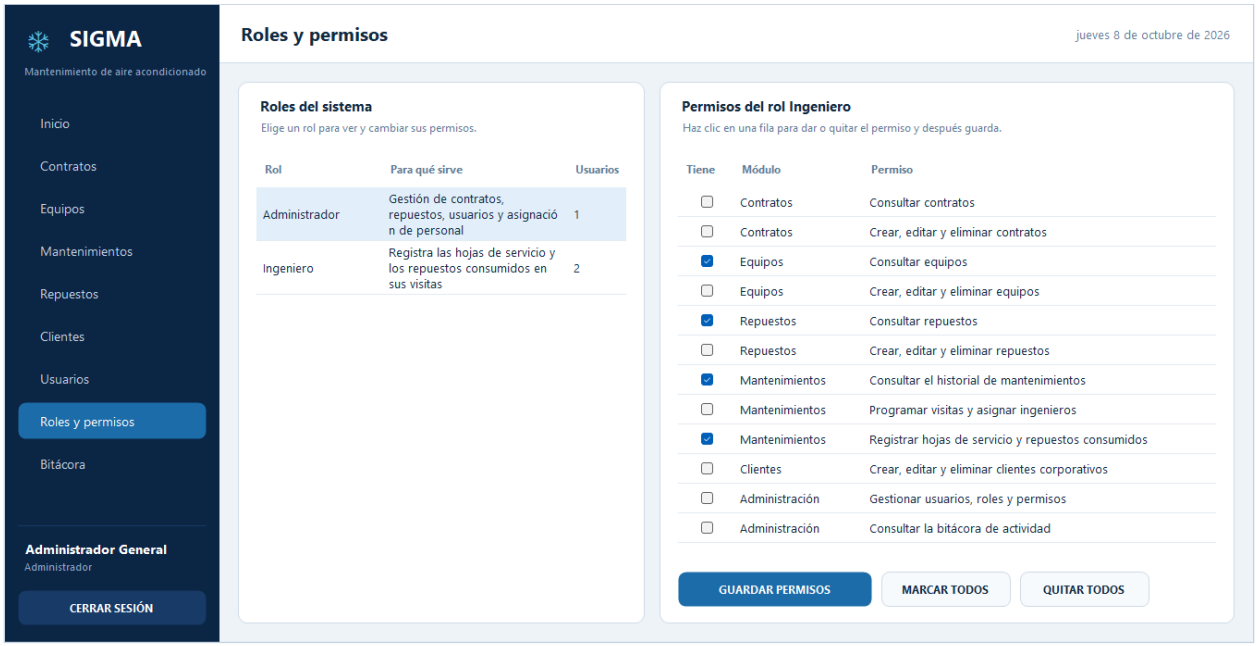



*Figura 8. Administrador: asignación de permisos a cada rol*

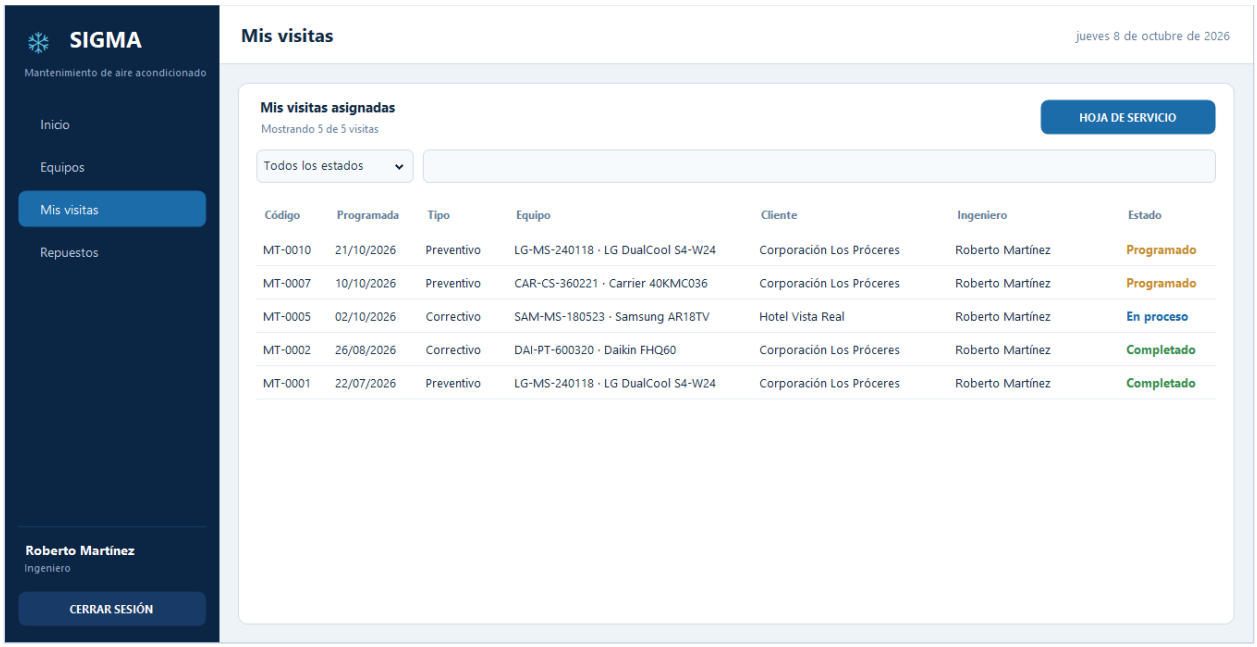



*Figura 9. Ingeniero: solo ve sus visitas asignadas y un menú reducido*

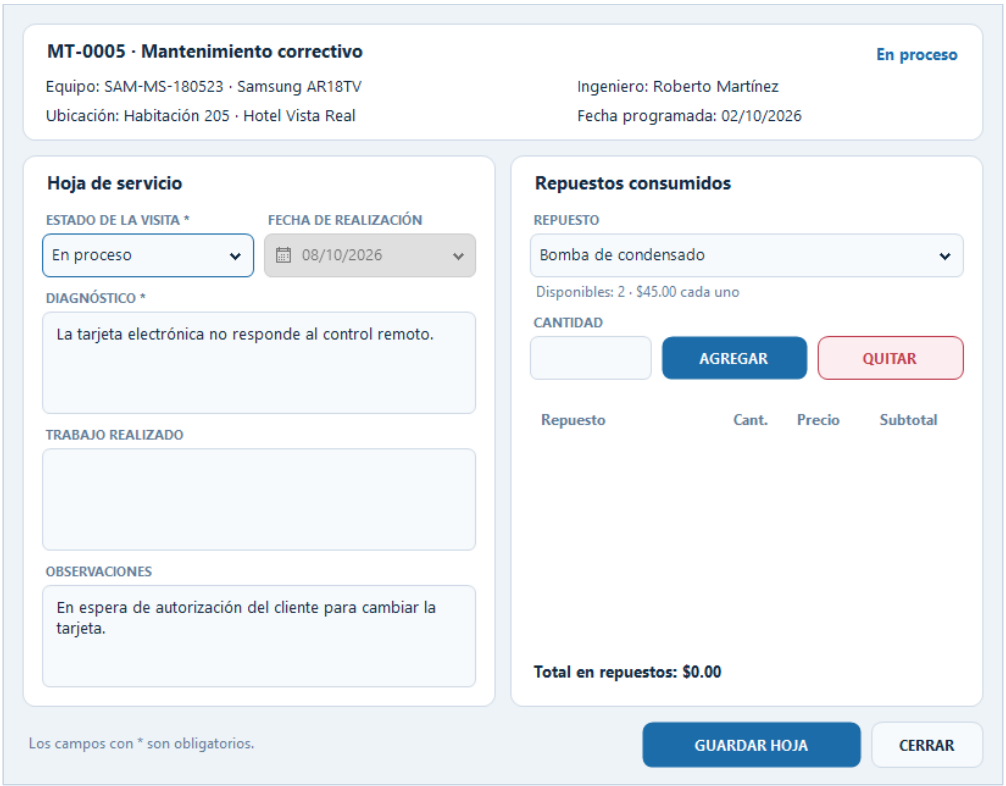



*Figura 10. Ingeniero: hoja de servicio con los repuestos consumidos*

# 11. Instalación y usuarios de prueba

## 11.1 Requisitos

- Windows 10 u 11 con .NET Framework 4.8.
- SQL Server Express (instancia localhost\SQLEXPRESS) y, de preferencia, SQL Server Management Studio.
- Visual Studio 2022 con la carga de trabajo de desarrollo de escritorio de .NET.

## 11.2 Pasos

1. Crear la base de datos: abrir **Modelo\Database\SIGMA_Script.sql** en SQL Server Management Studio y presionar Execute (F5), o hacer doble clic en **CrearBaseDeDatos.cmd** de esa misma carpeta.
2. Abrir **SIGMA.sln** en Visual Studio. Los paquetes BCrypt.Net-Next y Guna.UI2.WinForms se restauran solos al compilar (clic derecho en la solución, Restaurar paquetes NuGet).
3. Si el servidor no se llama localhost\SQLEXPRESS, cambiar el Data Source en **Vista\App.config**.
4. Establecer **Vista** como proyecto de inicio y presionar F5.

## 11.3 Usuarios de prueba

El script crea estos usuarios, con datos de ejemplo para probar cada rol:

| Usuario | Contraseña | Rol | Qué puede probar |
|---|---|---|---|
| **admin** | Admin123 | Administrador | Todo el sistema: contratos, equipos, repuestos, visitas, clientes, usuarios, roles y bitácora. |
| **ingeniero** | Ingeniero123 | Ingeniero | Sus visitas asignadas y el registro de hojas de servicio con repuestos. |
| **shernandez** | Ingeniero123 | Ingeniero | Segundo ingeniero, para comprobar que cada uno ve solo sus visitas. |